

mycv

Mathematical Foundations of a Pure-NumPy
Computer Vision Library

Abodunrin Charles Toluwanimi

Department of Electrical and Electronic Engineering
Obafemi Awolowo University, Ile-Ife

*A technical monograph documenting the complete mathematical derivations underlying each module
of mycv v4.0.0: a vectorised, dependency-free image processing and computer vision library
implemented in pure NumPy.*

Contents

1	Notation and Structural Conventions	3
2	Pointwise Operations (core.py)	3
2.1	RGB-to-Grayscale: ITU-R BT.601 Luminance	3
2.2	Binary Thresholding	3
3	Photometric Transformations (color.py)	4
3.1	Histogram Equalisation via CDF Remapping	4
3.2	RGB to HSV: Mapping the Colour Cube to the Cylinder	4
4	Spatial Filtering and Discrete Convolution (filters.py)	4
4.1	The Continuous Convolution Integral	4
4.2	2D Discrete Convolution	4
4.3	Sobel Edge Detection	5
5	Binary Morphology via Set Theory (morphology.py)	5
6	Geometric Transformations (geometry.py)	5
6.1	Centre-Rotation Matrix	5
6.2	Bilinear Interpolation	5
6.3	Projective Warping via Homography	6
7	Classical Feature Extraction (features.py)	6
7.1	Harris Corner Detector	6
7.2	Hough Line Transform	6
8	Object Detection Primitives (detection.py)	6
8.1	Normalised Cross-Correlation	6
8.2	Gaussian Image Pyramid	7
8.3	Non-Maximum Suppression	7
9	Motion Detection and Object Tracking (tracking.py)	7
9.1	Motion Detection via Frame Differencing	7
9.1.1	The uint8 Underflow Problem	7
9.1.2	Binary Motion Mask	7
9.2	Colour-Based Object Tracking	8
9.2.1	HSV Colour Range Mask	8
9.2.2	2-D Centroid via Discrete Expected Value	8
9.3	Temporal Smoothing via Circular Buffer	8
9.3.1	Temporal Noise and the Averaging Filter	8
9.3.2	Circular Buffer vs. Naive List Append	9
9.3.3	Overflow-Safe Temporal Average	9
10	Implementation Notes and Numerical Considerations	10
10.1	Precision and Overflow	10
10.2	Vectorisation Hierarchy	10
10.3	Module Dependency Graph	10
11	Summary of Key Formulae	11

References

12

1. Notation and Structural Conventions

Throughout this document an *image* is a function $I : \Omega \rightarrow \mathcal{V}$, $\Omega = \{0, \dots, H-1\} \times \{0, \dots, W-1\} \subset \mathbb{Z}^2$, where $H, W \in \mathbb{N}$ are the height and width and $\mathcal{V}$ is the value domain. For a grayscale image $\mathcal{V} = \{0, \dots, 255\} \cong \mathbb{Z}_{256}$; for a colour image $\mathcal{V} = \mathbb{Z}_{256}^3$. Row index y increases downward; column index x increases rightward. Total pixel count is $N = HW$. A *kernel* of size $k_H \times k_W$ is a matrix $K \in \mathbb{R}^{k_H \times k_W}$, with half-extents $p = \lfloor k_H/2 \rfloor$, $q = \lfloor k_W/2 \rfloor$. Homogeneous coordinates augment $\mathbf{x} = (x, y)^\top \in \mathbb{R}^2$ to $\tilde{\mathbf{x}} = (x, y, 1)^\top \in \mathbb{R}^3$.

Module map.

Module	Mathematical domain	Section
<code>core.py</code>	Pointwise linear algebra	§2
<code>color.py</code>	Probability theory; coordinate geometry	§3
<code>filters.py</code>	Discrete convolution; differential operators	§4
<code>morphology.py</code>	Set theory; lattice operations	§5
<code>geometry.py</code>	Affine & projective geometry; interpolation	§6
<code>features.py</code>	Spectral methods; accumulator geometry	§7
<code>detection.py</code>	Statistical correlation; sampling theory; IoU	§8
<code>tracking.py</code>	Unsigned arithmetic; probability; ring buffers	§9

2. Pointwise Operations (`core.py`)

2.1 RGB-to-Grayscale: ITU-R BT.601 Luminance

The human visual system does not respond uniformly to equal-energy stimuli at different wavelengths. The three cone photoreceptor classes — L-cones (≈ 565 nm), M-cones (≈ 540 nm), and S-cones (≈ 445 nm) — have markedly different foveal densities, with S-cones comprising fewer than 10% of the total.

Definition 2.1 (BT.601 Luminance). Let $I_{\text{RGB}} : \Omega \rightarrow [0, 255]^3$ be an RGB image. The BT.601 luminance of pixel (x, y) is

$$Y(x, y) = \mathbf{w}^\top \begin{pmatrix} R(x, y) \\ G(x, y) \\ B(x, y) \end{pmatrix}, \quad \mathbf{w} = (0.2989, 0.5870, 0.1140)^\top. \quad (1)$$

$\|\mathbf{w}\|_1 = 1$ ensures $Y(x, y) \in [0, 255]$. The implementation is a single BLAS-accelerated rank-1 contraction: `np.dot(image.astype(float64), w)`.

Remark 2.2. ITU-R BT.709 (HDTV) uses $\mathbf{w}_{709} = (0.2126, 0.7152, 0.0722)^\top$.

2.2 Binary Thresholding

Definition 2.3 (Global Binary Threshold).

$$T_\tau(I)(x, y) = 255 \cdot \mathbf{1}_{[I(x, y) \geq \tau]}. \quad (2)$$

The mask $F_\tau = \{(x, y) \in \Omega : I(x, y) \geq \tau\}$ partitions Ω . Implementation: `(I >= tau).astype(uint8) * 255` — one vectorised comparison, no branching.

3. Photometric Transformations (color.py)

3.1 Histogram Equalisation via CDF Remapping

Definition 3.1 (Histogram and PDF). $h(k) = |\{(x, y) \in \Omega : I(x, y) = k\}|$; $p(k) = h(k)/N$.

Definition 3.2 (CDF and Equalisation Map).

$$F(k) = \sum_{j=0}^k p(j), \quad T(k) = \text{round } 255 \cdot F(k). \quad (3)$$

Derivation. For a continuous variable X with CDF F_X , $Y = 255F_X(X)$ has CDF $F_Y(y) = y/255$, the uniform CDF on $[0, 255]$. The discrete version approximates this by rounding, maximising the spread of occupied intensity levels. $\square$

Pipeline: $h \xrightarrow{\div N} p \xrightarrow{\text{prefix sum}} F \xrightarrow{\times 255, \text{round}} T \xrightarrow{T[I]} I_{\text{eq}}$

3.2 RGB to HSV: Mapping the Colour Cube to the Cylinder

Let $r, g, b \in [0, 1]$, $V = \max(r, g, b)$, $C_{\min} = \min(r, g, b)$, $\Delta = V - C_{\min}$.

Definition 3.3 (HSV Components).

$$S = \begin{cases} \Delta/V & V \neq 0 \\ 0 & V = 0, \end{cases} \quad (4)$$

$$H = \begin{cases} 60((g - b)/\Delta) \bmod 360 & V = r, \\ 60((b - r)/\Delta + 2) & V = g, \\ 60((r - g)/\Delta + 4) & V = b, \\ 0 & \Delta = 0. \end{cases} \quad (5)$$

All three hue formulae are computed simultaneously across the full $(H \times W)$ grid and selected via Boolean masks — no piecewise Python branching.

4. Spatial Filtering and Discrete Convolution (filters.py)

4.1 The Continuous Convolution Integral

$(f * g)(\mathbf{x}) = \iint_{\mathbb{R}^2} f(\tau\mathbf{0}) g(\mathbf{x} - \tau\mathbf{0}) d\tau$. The Convolution Theorem: $(f * g)^\wedge = \hat{f} \cdot \hat{g}$, so applying a kernel K is a linear, shift-invariant filter with transfer function $\hat{K}$.

4.2 2D Discrete Convolution

Definition 4.1 (Discrete 2D Convolution).

$$(I * K)[x, y] = \sum_{i=-p}^p \sum_{j=-q}^q I[x - i, y - j] \cdot K[i, j]. \quad (6)$$

Remark 4.2 (Convolution vs. Cross-correlation). Cross-correlation replaces $I[x - i, y - j]$ with $I[x + i, y + j]$. mycv implements true convolution via `np.flip(K)` before contraction.

The zero-copy patch tensor $\mathcal{P} \in \mathbb{R}^{H \times W \times k_H \times k_W}$ is built via `as_strided`; the contraction collapses to `np.einsum("hwij, ij->hw", P, K_flipped)`.

4.3 Sobel Edge Detection

Definition 4.3 (Sobel Kernels).

$$K_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} (-1 \ 0 \ +1), \quad K_y = K_x^\top. \quad (7)$$

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}, \quad \Theta(x, y) = \arctan\left(\frac{G_y}{G_x}\right) \in (-\pi, \pi]. \quad (8)$$

5. Binary Morphology via Set Theory (morphology.py)

A binary image is identified with its foreground set $F = \{(x, y) \in \Omega : I(x, y) = 1\}$.

Definition 5.1 (Dilation and Erosion).

$$F \oplus B = \{\mathbf{z} : \hat{B}_{\mathbf{z}} \cap F \neq \emptyset\} \quad (\text{ANY over patch}) \quad (9)$$

$$F \ominus B = \{\mathbf{z} : B_{\mathbf{z}} \subseteq F\} \quad (\text{ALL over patch}) \quad (10)$$

Definition 5.2 (Opening and Closing). $F \circ B = (F \ominus B) \oplus B$; $F \bullet B = (F \oplus B) \ominus B$.

Theorem 5.3 (Morphological Properties). *Idempotency; anti-extensivity of opening ($F \circ B \subseteq F$); extensivity of closing ($F \subseteq F \bullet B$); duality ($F \circ B = \overline{F \bullet \hat{B}}$).*

6. Geometric Transformations (geometry.py)

6.1 Centre-Rotation Matrix

A rotation by θ about the image centre $(c_x, c_y) = ((W-1)/2, (H-1)/2)$:

Definition 6.1 (Centre-Rotation).

$$M(\theta) = T_{\text{back}} \cdot R(\theta) \cdot T_{\text{origin}}, \quad R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (11)$$

Backward mapping via M^{-1} guarantees a dense, hole-free output.

6.2 Bilinear Interpolation

Let $x_0 = \lfloor x_s \rfloor$, $\delta_x = x_s - x_0$ (similarly for y).

Definition 6.2 (Bilinear Interpolation).

$$I(x_s, y_s) = (1-\delta_x)(1-\delta_y)I[y_0, x_0] + \delta_x(1-\delta_y)I[y_0, x_1] + (1-\delta_x)\delta_y I[y_1, x_0] + \delta_x\delta_y I[y_1, x_1]. \quad (12)$$

Error is $\mathcal{O}(h^2)$ for grid spacing $h = 1$. Out-of-bounds coordinates are clamped to the image boundary.

6.3 Projective Warping via Homography

Definition 6.3 (Projective Division). For $\mathbf{q} = H^{-1}\tilde{\mathbf{p}}' = (\tilde{x}, \tilde{y}, \tilde{w})^\top$:

$$x_s = \tilde{x}/\tilde{w}, \quad y_s = \tilde{y}/\tilde{w}. \quad (13)$$

Theorem 6.4 (Necessity of Projective Division). *Division by $\tilde{w}$ is the unique non-linear operation encoding depth foreshortening; it is necessary and sufficient for parallel-to-convergent line mapping. An affine homography ($\tilde{w} \equiv 1$) cannot produce perspective convergence.*

7. Classical Feature Extraction (features.py)

7.1 Harris Corner Detector

Definition 7.1 (Structure Tensor and Corner Response).

$$M(x, y) = \sum_W w \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}, \quad R = \det M - k(\text{tr } M)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2. \quad (14)$$

$k \in [0.04, 0.06]$ (default 0.05). The windowed sum is implemented as convolution with a Gaussian kernel G_σ via the existing `convolve2d` engine.

Proposition 7.2 (Eigenvalue Classification). $\lambda_1 \approx \lambda_2 \gg 0$: *corner*. $\lambda_1 \gg \lambda_2 \approx 0$: *edge*. $\lambda_1 \approx \lambda_2 \approx 0$: *flat*.

7.2 Hough Line Transform

Definition 7.3 (Hough Polar Form).

$$\rho = x \cos \theta + y \sin \theta, \quad \theta \in [-\pi/2, \pi/2], \quad \rho \in [-d, d]. \quad (15)$$

All NT votes are computed in a single $(N \times T)$ broadcast then accumulated with a single `np.add.at` call. Complexity: $\mathcal{O}(NT)$ time, $\mathcal{O}(NT + 2dT)$ space.

8. Object Detection Primitives (detection.py)

8.1 Normalised Cross-Correlation

Raw cross-correlation $\text{CC}(x, y) = \sum_{i,j} I[x+i, y+j] \cdot T[i, j]$ is sensitive to absolute intensity. The normalised variant removes this dependence by applying Pearson's correlation coefficient.

Definition 8.1 (NCC Response Map).

$$\text{NCC}(x, y) = \frac{\frac{1}{k_H k_W} \sum_{i,j} (P_{xy}[i, j] - \mu_P)(T[i, j] - \mu_T)}{\sigma_P \sigma_T}. \quad (16)$$

Proposition 8.2 (Affine Invariance). *NCC is invariant under $X \mapsto aX + b$, $Y \mapsto cY + d$ ($a, c > 0$). The patch tensor $\mathcal{P}$ is built via `as_strided`; all statistics reduce over axes $(-2, -1)$ in three vectorised calls.*

Cosine similarity equivalence

Rewrite the zero-mean patch $\tilde{P}_{xy}$ as a vector in $\mathbb{R}^{k_H k_W}$. Then $\text{NCC}(x, y) = \langle \tilde{P}_{xy}, \tilde{T} \rangle / (\|\tilde{P}_{xy}\| \|\tilde{T}\|) = \cos \angle(\tilde{P}_{xy}, \tilde{T})$. Flat patches ($\sigma_P = 0$) receive $\text{NCC} = 0$ via `np.where`.

8.2 Gaussian Image Pyramid

Definition 8.3 (Gaussian Pyramid). $I_0 = I$; for $k \geq 1$:

$$I_k = (I_{k-1} * G_{5 \times 5})[:, 2 :: 2]. \quad (17)$$

Downsampling without prior blurring causes aliasing: energy at $f \in (f'_N, f_N)$ folds back into $[0, f'_N]$ irreversibly (Nyquist–Shannon theorem). The 5×5 binomial Gaussian kernel attenuates those frequencies before decimation. Total pyramid storage: $\leq \frac{4}{3}HW$.

8.3 Non-Maximum Suppression

Definition 8.4 (IoU / Jaccard Index).

$$\text{IoU}(A, B) = \frac{\mathcal{J}(A, B)}{\mathcal{A}(A) + \mathcal{A}(B) - \mathcal{J}(A, B)}, \quad (18)$$

$$\mathcal{J}(A, B) = \max(0, y_2^I - y_1^I) \cdot \max(0, x_2^I - x_1^I).$$

Greedy NMS iteratively selects the highest-scoring box and suppresses candidates with $\text{IoU} > \tau$ in a single vectorised broadcast per iteration.

9. Motion Detection and Object Tracking (tracking.py)

9.1 Motion Detection via Frame Differencing

9.1.1 The uint8 Underflow Problem

Pixel intensities are stored as unsigned 8-bit integers, meaning $I[y, x] \in \{0, \dots, 255\} \cong \mathbb{Z}_{256}$. Arithmetic on uint8 arrays wraps modulo 256:

$$(10_{\text{uint8}}) - (200_{\text{uint8}}) \equiv 10 - 200 + 256 = 66_{\text{uint8}},$$

whereas the true absolute difference is 190. Whenever `frame_t[y,x] < frame_t_minus_1[y,x]`, the direct subtraction produces a small positive value — silently transforming real motion into an apparent non-event.

Definition 9.1 (Safe Absolute Frame Difference). Cast both frames to signed int16 (range $[-32768, 32767]$) before subtracting, then take the absolute value:

$$D(x, y) = |I_t(x, y) - I_{t-1}(x, y)|, \quad I_t, I_{t-1} \in \{0, \dots, 255\}, D \in \{0, \dots, 255\}. \quad (19)$$

The maximum value $|255 - 0| = 255$ fits comfortably in int16.

Vectorised implementation.

$$D = \text{np.abs}(\text{frame_t.astype(int16)} - \text{frame_t_minus_1.astype(int16)})$$

One cast, one subtraction, one `np.abs` — no Python loop over pixels.

9.1.2 Binary Motion Mask

The motion mask applies the same thresholding function as (2):

$$\text{Motion}(x, y) = 255 \cdot \mathbf{1}_{[D(x,y) \geq \tau]}, \quad (20)$$

where τ is the motion sensitivity threshold. Pixel (x, y) is declared moving iff the inter-frame intensity change exceeds τ . The implementation is `(D >= threshold).astype(uint8) * 255` — a single vectorised comparison with no branching.

9.2 Colour-Based Object Tracking

9.2.1 HSV Colour Range Mask

HSV space separates the achromatic (brightness) and chromatic (hue, saturation) components of colour, making thresholding under varying illumination more robust than thresholding in RGB.

Definition 9.2 (HSV Colour Mask). Given an HSV image $\mathbf{I}_{\text{HSV}} : \Omega \rightarrow [0, 360) \times [0, 1]^2$ and bounds $\ell, u \in \mathbb{R}^3$, the colour mask is

$$M(x, y) = 255 \cdot \mathbf{1}_{[\ell_H \leq H \leq u_H]} \cdot \mathbf{1}_{[\ell_S \leq S \leq u_S]} \cdot \mathbf{1}_{[\ell_V \leq V \leq u_V]}. \quad (21)$$

A pixel is foreground iff its HSV triple simultaneously satisfies all three closed-interval membership conditions. This is a query against the axis-aligned 3-D box $[\ell, u]$ in HSV space.

Equation (21) is implemented as a single vectorised bitwise-AND of three boolean arrays:

$$M = (H \geq \ell_H) \ \& \ (H \leq u_H) \ \& \ (S \geq \ell_S) \ \& \ (S \leq u_S) \ \& \ (V \geq \ell_V) \ \& \ (V \leq u_V).$$

Remark 9.3 (Hue Wrap-Around). Hue is angular and periodic. Red spans both $H \approx 0^\circ$ and $H \approx 360^\circ$. When the target range wraps (i.e. $\ell_H > u_H$), compute two non-wrapping masks and combine with bitwise OR: $M = \text{color_mask}(\dots, \ell_H, 360) \mid \text{color_mask}(\dots, 0, u_H)$.

9.2.2 2-D Centroid via Discrete Expected Value

Definition 9.4 (2-D Centroid). Given the foreground set $F = \{(x, y) \in \Omega : M(x, y) > 0\}$, the *centroid* (centre of mass) is:

$$c_x = \frac{1}{|F|} \sum_{(x,y) \in F} x = \mathbb{E}_F[X], \quad c_y = \frac{1}{|F|} \sum_{(x,y) \in F} y = \mathbb{E}_F[Y]. \quad (22)$$

This is the discrete 2-D analogue of the first raw moment normalised by the zeroth moment (total mass), equivalent to the arithmetic mean coordinate of all foreground pixels. The relationship to continuous probability theory is exact: treating the uniform distribution over F as a 2-D random variable (X, Y) , the centroid is its expected value.

Proposition 9.5 (Centroid from `np.nonzero`). `np.nonzero(M)` returns arrays `ys`, `xs` of length $|F|$ containing the row and column indices of all foreground pixels. Then:

$$c_x = \text{np.mean(xs)}, \quad c_y = \text{np.mean(ys)}.$$

This runs in $\mathcal{O}(|F|)$ with no Python-level pixel loop. When $|F| = 0$ (entirely black mask), the centroid is undefined; the function returns $(-1, -1)$ as a sentinel value rather than raising `ZeroDivisionError`.

9.3 Temporal Smoothing via Circular Buffer

9.3.1 Temporal Noise and the Averaging Filter

Individual frames from a digital sensor contain *temporal noise*: independent zero-mean random fluctuations in each pixel caused by photon shot noise, readout noise, and quantisation error. Let $\eta_t(x, y) \sim \mathcal{N}(0, \sigma^2)$ be the noise component of frame t . Averaging N independent frames suppresses this noise by $1/\sqrt{N}$:

$$\bar{I}(x, y) = \frac{1}{N} \sum_{k=0}^{N-1} I_{t-k}(x, y) \Rightarrow \text{Var}[\bar{I}(x, y)] = \frac{\sigma^2}{N}. \quad (23)$$

For a live feed, only the most recent N frames are available and a new frame arrives at every timestep. An efficient implementation must add a new frame and discard the oldest in $\mathcal{O}(1)$ time.

9.3.2 Circular Buffer vs. Naive List Append

Consider two implementations updating a running window of N frames, each of shape (H, W, C) .

Naive approach (list.pop):

Operation	Cost	Explanation
<code>frames.append(f)</code>	$\mathcal{O}(HWC)$	copy new frame
<code>frames.pop(0)</code>	$\mathcal{O}(N)$	shift $N - 1$ pointers forward
<code>np.mean(frames)</code>	$\mathcal{O}(NHC)$	reduce all N frames

Circular buffer approach:

Operation	Cost	Explanation
<code>buf[t%N] = f</code>	$\mathcal{O}(HWC)$	single in-place write
<code>t += 1</code>	$\mathcal{O}(1)$	integer increment
<code>np.mean(buf)</code>	$\mathcal{O}(NHC)$	reduce pre-allocated tensor

The `list.pop(0)` operation requires the Python runtime to shift $N - 1$ internal pointers forward — an $\mathcal{O}(N)$ memory operation on every single frame. For $N = 10$ frames of 1080p RGB video (≈ 6.2 MB per frame) this is ~ 62 MB of pointer bookkeeping per update, entirely independent of actual image data processing.

Definition 9.6 (Circular Buffer Ring Index). Pre-allocate a 4-D tensor $B \in \mathbb{R}^{N \times H \times W \times C}$ and maintain a non-decreasing counter $t \in \mathbb{N}_0$ (total frames written). The write index is

$$k = t \bmod N, \quad (24)$$

which cycles through $\{0, \dots, N - 1\}$ indefinitely via modulo arithmetic. Frame insertion is:

$$B[k] \leftarrow I_t \quad (\text{single float32 array write}).$$

Proposition 9.7 (Properties of the Ring Buffer). *(i) The slot at index k always holds the most recently written frame.*

(ii) The slot at index $(k + 1) \bmod N$ always holds the oldest retained frame.

(iii) No data is ever moved; only $B[k]$ is overwritten.

(iv) Write cost is $\mathcal{O}(HWC)$ (one array assignment), with $\mathcal{O}(1)$ index arithmetic overhead.

9.3.3 Overflow-Safe Temporal Average

The temporal average is:

$$\bar{I}(x, y) = \frac{1}{N} \sum_{k=0}^{N-1} B[k, y, x], \quad (25)$$

implemented as `buffer.mean(axis=0)`. This must be computed in a wider numeric type. Since $B \in \{0, \dots, 255\}^{N \times H \times W \times C}$, a partial sum can reach $255N$, which exceeds the `uint8` maximum of 255 for any $N > 1$. The buffer is therefore stored in `float32` (maximum exact integer: $2^{24} \approx 16.7 \times 10^6$, far exceeding $255N$ for any practical N), and the result is clipped and cast back to `uint8` only after the division:

$$\text{output} = \text{clip}(\text{mean}_{\text{float32}}(B), 0, 255). \text{astype}(\text{uint8}).$$

Before the buffer is fully populated (fewer than N frames written), only the valid slots $B[0 : \min(t, N)]$ are averaged.

10. Implementation Notes and Numerical Considerations

10.1 Precision and Overflow

All intermediate computations are performed in `float64` (or `float32` for the ring buffer, where 2^{24} exceeds $255N$ for all practical N). Inputs in `uint8` are upcast before any arithmetic; outputs are clipped to $[0, 255]$ and downcast to `uint8` only at function boundaries. The `uint8` underflow problem in frame differencing is resolved by casting to `int16` before subtraction (§9.1).

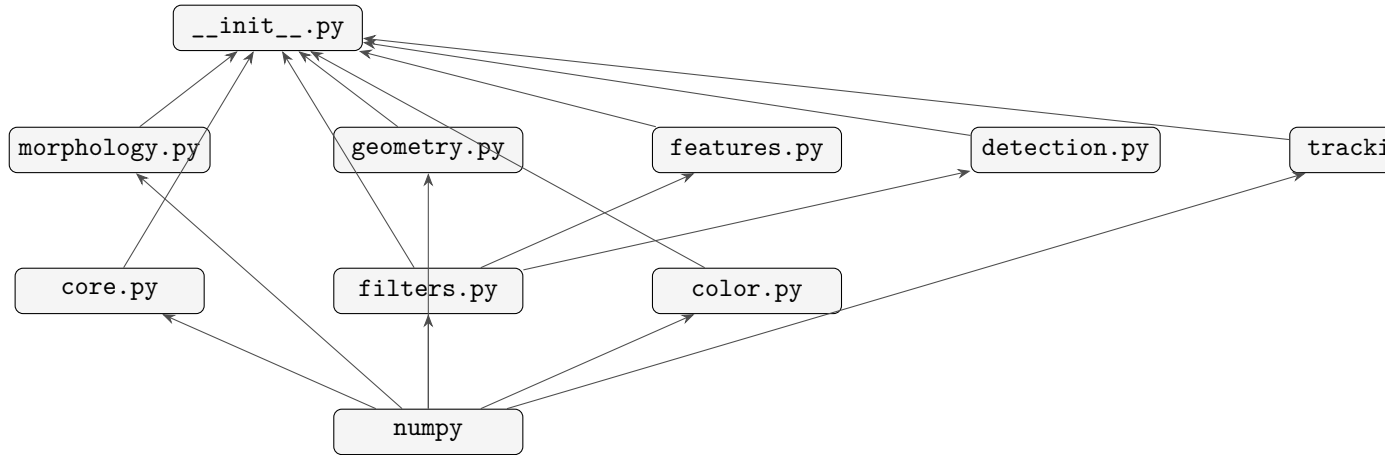
10.2 Vectorisation Hierarchy

Every function in `mycv` obeys the following priority ordering:

1. *Single-pass broadcast*: one NumPy expression over full arrays — `np.dot`, `lut[image]`, `np.sqrt(Gx**2+Gy**2)`, frame differencing with `np.abs`.
2. *Stride-trick + einsum/reduction*: zero-copy patch tensor contracted by `np.einsum` or reduced by `np.any/np.all` — `convolve2d`, `dilate`, `erode`, `match_template_ncc`.
3. *Scatter with `np.add.at`*: Hough accumulator, where repeated index increments must not be silently collapsed.
4. *Masked assignment via `np.where`*: HSV hue selection, NCC zero-variance guard, IoU union guard, motion mask threshold.

No Python-level `for` loop iterates over individual pixels.

10.3 Module Dependency Graph



11. Summary of Key Formulae

Operation	Formula	Eq.
BT.601 Luminance	$Y = 0.2989R + 0.5870G + 0.1140B$	(1)
Threshold	$T_\tau(I)(x, y) = 255 \cdot \mathbf{1}_{[I \geq \tau]}$	(2)
CDF equalisation	$T(k) = \text{round } 255 \cdot \sum_{j=0}^k p(j)$	(3)
HSV Saturation/Hue	$S = \Delta/V$; H piecewise in $[0^\circ, 360^\circ)$	(4)–(5)
Discrete convolution	$(I * K)[x, y] = \sum_{i,j} I[x-i, y-j] K[i, j]$	(6)
Sobel magnitude	$M = \sqrt{G_x^2 + G_y^2}$	(8)
Dilation / Erosion	$F \oplus B$: ANY; $F \ominus B$: ALL	(9)–(10)
Centre-rotation	$M(\theta) = T_{\text{back}} \cdot R(\theta) \cdot T_{\text{origin}}$	(11)
Bilinear interp.	$I(x_s, y_s) = \sum_{k \in \{0,1\}^2} w_k I[y_k, x_k]$	(12)
Projective division	$x_s = \tilde{x}/\tilde{w}$, $y_s = \tilde{y}/\tilde{w}$	(13)
Harris response	$R = \det M - k(\text{tr } M)^2$	(14)
Hough polar line	$\rho = x \cos \theta + y \sin \theta$	(15)
NCC (Pearson)	$\text{NCC} = \text{cov}(P, T) / (\sigma_P \sigma_T)$	(16)
Gaussian pyramid	$I_k = (I_{k-1} * G)[::2, ::2]$	(17)
IoU (Jaccard)	$\text{IoU} = \mathcal{I} / (\mathcal{A}(A) + \mathcal{A}(B) - \mathcal{I})$	(18)
Safe frame diff	$D = I_t^{\text{int16}} - I_{t-1}^{\text{int16}} $	(19)
Motion mask	$\text{Motion}(x, y) = 255 \cdot \mathbf{1}_{[D \geq \tau]}$	(20)
HSV colour mask	$M = \mathbf{1}_{[\ell_H, u_H]}(H) \cdot \mathbf{1}_{[\ell_S, u_S]}(S) \cdot \mathbf{1}_{[\ell_V, u_V]}(V)$	(21)
Centroid	$c_x = \mathbb{E}_F[X]$, $c_y = \mathbb{E}_F[Y]$	(22)
Ring buffer index	$k = t \bmod N$	(24)
Temporal average	$\bar{I} = \frac{1}{N} \sum_{k=0}^{N-1} B[k]$	(25)

References

- [1] C. Harris and M. Stephens, “A combined corner and edge detector,” *Proc. Alvey Vision Conf.*, 1988, pp. 147–151.
- [2] P. V. C. Hough, “Method and means for recognizing complex patterns,” U.S. Patent 3,069,654, 1962.
- [3] ITU-R Recommendation BT.601-7, *Studio Encoding Parameters of Digital Television*, 2011.
- [4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [5] G. Matheron, *Random Sets and Integral Geometry*. Wiley, 1975.
- [6] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [7] S. G. Johnson, “Notes on the Hough Transform,” Massachusetts Institute of Technology, 2011.
- [8] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *Int. J. Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [9] P. Viola and M. Jones, “Rapid object detection using a boosted cascade,” *Proc. CVPR*, vol. 1, pp. 511–518, 2001.
- [10] C. E. Shannon, “Communication in the presence of noise,” *Proc. IRE*, vol. 37, no. 1, pp. 10–21, 1949.
- [11] J. P. Lewis, “Fast normalized cross-correlation,” *Vision Interface*, vol. 10, no. 1, pp. 120–123, 1995.
- [12] A. Papoulis, *Probability, Random Variables, and Stochastic Processes*, 3rd ed. McGraw-Hill, 1991.